

Chapter 59. Industry Case Studies

Nine end-to-end projects read through one frame: the problem, the data, the approach, the architecture, the result, and what went wrong.

Draft chapter

Contents

Chapter 59. Industry Case Studies

Why this matters

In plain English

How to read a case study

Case 1 · Manufacturing quality: the defect camera

Case 2 · Demand planning at a mid-sized FMCG distributor

Case 3 · B2B lead scoring at a software company

Case 4 · Retail pricing at a regional chain

Case 5 · Fraud detection at a bank

Case 6 · Logistics routing for a distribution fleet

Case 7 · Customer support automation at a subscription business

Case 8 · Predictive maintenance on plant machinery

Case 9 · Order to cash, end to end

What the nine have in common

What changes by industry, and what doesn't

In the real world: how these projects actually start

Common mistakes and how to spot them

Recap

Key terms

Where this leads

Chapter 59. Industry Case Studies

Part VI — Production ML, Generative AI & MLOps

Chapter at a glance

You will learn to: recognize the shape every data project shares, whatever the industry · read a case study for its constraints rather than its technology · see where the real work sits in nine different problems, and why it is so rarely the model · name the mistake each one made, what it cost, and how it was fixed · and judge your own next project against patterns that have already been through production.

Before you start: nothing new. This chapter draws on everything from Part II onwards, and each case names the chapters that built its pieces.

Time needed: 3–4 hours to read, and a useful afternoon to map your own work onto the frame.

Tools and practice data: none. This is a reading chapter.

Why this matters

Six parts of this book have taught pieces: SQL, Python, statistics, modeling, pipelines, models in production. A project is not a list of pieces, and the gap between "I can do each of these" and "I have delivered one of these" is where most careers stall for a year or two.

Case studies close that gap, if they are read properly. Read badly they are technology tourism: *they used a transformer, we should use a transformer*. Read well, they are a catalogue of **constraints**, what the data actually contained, who had to act on the output, what an error cost, how late the truth arrived, and constraints are what transfer between industries. A fraud team and a maintenance team have almost nothing in common technically and exactly the same problem: **rare events, late labels, and a fixed amount of human attention to spend**.

This chapter reads nine projects that way. Three are Riverstone's, and their numbers were measured in this book, in chapters you have already worked through. The other six are composites: patterns as they typically run, with representative numbers rather than measured ones, labeled as such every time.

In plain English

Every data project is the same six questions, asked in a different order.

1. **What decision changes?** If no decision changes, there is no project, however interesting the data.

2. **What data exists, and what has to be created?** The second half is usually the bigger half, and it is usually labels.
3. **What is the simplest thing that could work?** Whatever you answer, the honest version is simpler.
4. **Where does the output go?** A screen, a queue, a system of record, or an automated action, in rising order of danger.
5. **What does an error cost, and who finds it?** This sets your threshold, your review capacity, and whether you should automate at all.
6. **Who owns it in a year?** Projects without an answer here are demos with a longer runway.

The nine cases below differ in every technical particular and answer the same six questions. That is the thing worth taking away: **the frame transfers; the technology doesn't.**

The six questions every project in this chapter answers

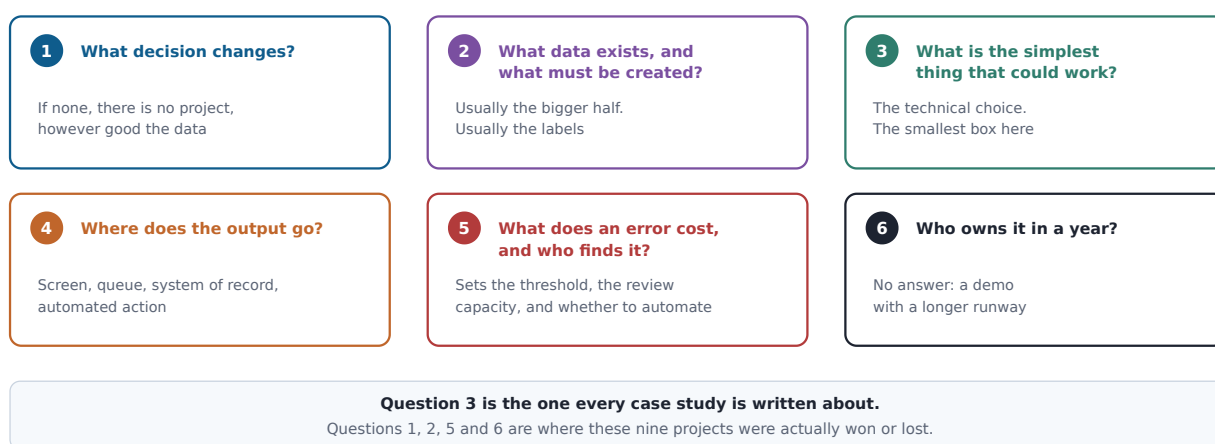


Figure 59.1 – The frame every case in this chapter is read through. The technical choices sit inside question three, which is the smallest box.

How to read a case study

Four habits, worth practicing on the nine below and on the next vendor deck you are sent.

- **Find the constraint before the technology.** "We used gradient boosting" is not information; "we had 300 labeled failures over two years" is, and it explains everything downstream.
- **Ask what the baseline was.** A model that beats nothing is not a result. The most common missing number in published case studies is what the business was already achieving.
- **Ask when the truth arrives.** A system whose ground truth lands in six weeks (maintenance, fraud recovery, churn) is a different engineering problem from one that knows within a second (a click).
- **Read the deployment sentence.** Almost every case study describes the model in three paragraphs and the deployment in one clause. The clause is where the project's real difficulty lives, and the absence of one usually means it never shipped.

Watch out: published case studies are marketing. The ones with vendor logos are written to sell a product, the numbers are chosen after the fact, and the failures are absent. The nine here include what went wrong on purpose, because a chapter of nine successes would teach nothing except that success is normal, which it is not.

Case 1 • Manufacturing quality: the defect camera

Riverstone's own, measured in Chapters 53 and 56.

The problem as stated	"Can AI check our parts so we don't need an inspector on line two?"
The decision that changes	Whether a moulded lid goes to dispatch or to the reject bin
The data	6,000 camera images, 7.9% defective, three fault types; labels came from the QC inspector's existing log
The approach	Convolution features into a small classifier (Chapter 53); threshold chosen from the cost of a miss against a false alarm
The architecture	Camera → feature extraction → model behind a small HTTP service → verdict on the line's screen → 2% audit of passed parts → weekly retraining review (Chapter 56)
The result	97.5% of defects caught at a threshold that costs ₹13,840 per 1,500 parts, against ₹40,080 at the default threshold
What went wrong	Trained on day-shift images only. On the night shift the lamps are dimmer, and a model that had learned the contrast at the rim started passing short shots. Found by a customer return, six weeks later
The fix	Night-shift images in training, brightness augmentation, an input-brightness monitor, and the 2% audit that makes recall measurable in a week rather than a quarter
The lesson	Accuracy was never the metric. The project's real deliverable was an operating point priced in rupees, and a monitoring plan that measured the thing the drift monitor could not see

Case 2 • Demand planning at a mid-sized FMCG distributor

A composite; numbers are representative of this pattern rather than measured.

The problem as stated	"Our forecasts are wrong; we need machine learning"
The decision that changes	How much of each SKU to hold at each depot, ordered six weeks ahead
The data	Three years of daily despatch, promotions in a spreadsheet nobody owned, and a calendar of festivals that existed only in the sales head's memory
The approach	A statistical baseline first (Chapter 19's seasonal model), then gradient boosting on features that included promotions and festivals, with the forecast produced per SKU-depot-week

The architecture	Warehouse tables → feature build in dbt (Chapter 32) → weekly batch scoring → forecasts written back to a planning table → planners adjust in the ERP
The result	Forecast error down by roughly a quarter against the naive seasonal baseline; the meaningful result was three weeks of stock released because safety stock could be cut where the forecast was reliable
What went wrong	The first six months changed nothing. Planners did not trust a number with no explanation and kept their spreadsheets. Model accuracy was never the obstacle
The fix	Show the drivers beside the forecast (" +18% for Diwali, −5% trend"), show last month's accuracy per SKU, and let planners override with a recorded reason. Overrides became the best feature-engineering backlog the team had
The lesson	A forecast nobody acts on is a report. Half the project was adoption, and the tell was that accuracy improved for six months while nothing downstream moved

Case 3 • B2B lead scoring at a software company

A composite; representative numbers.

The problem as stated	"Score our leads so sales calls the good ones first"
The decision that changes	Which 40 of 400 weekly inbound leads a rep calls on day one
The data	CRM records entered by 30 salespeople over five years: free-text industries, duplicate accounts, and an outcome field that was only filled in when someone remembered
The approach	Six weeks of data work (Chapter 14): deduplication, industry normalization, and reconstructing outcomes from activity history. Then logistic regression, which was chosen because reps had to be told <i>why</i> a lead scored well
The architecture	CRM → nightly extract → feature build → scoring → score and top three reasons written back to the CRM record
The result	Conversion on called leads up by about a third; time to first contact for high-scoring leads down from days to hours
What went wrong	The first model scored brilliantly in testing and uselessly in production. It had learned from fields that are only filled in <i>after</i> a lead converts (Chapter 38's leakage), so it was predicting the past
The fix	A strict rule that every feature must be reconstructable from what was known at the moment the lead arrived, enforced by building the training set from time-stamped snapshots rather than the current record
The lesson	The model was three days of the project and the CRM data was six weeks. That ratio is normal, and quoting it early is how you get the six weeks funded

Case 4 • Retail pricing at a regional chain

A composite; representative numbers.

The problem as stated	"Find the optimal price for every product"
The decision that changes	The shelf price of about 2,000 products, reviewed monthly
The data	Two years of transactions, competitor prices scraped weekly, and a price history in which almost every change had been made <i>because</i> something else was happening
The approach	Elasticity estimated from historical price changes, then stopped, because the estimates were confounded (Chapter 31). The project switched to running actual price experiments in matched store groups (Chapter 30)
The architecture	Point of sale → warehouse → elasticity estimates per category → an experiment platform for store-group tests → approved changes into the pricing system, with a rule engine enforcing floors and competitive limits
The result	Margin up around 2% on the tested categories, with two categories where the test showed the proposed increase would have lost money
What went wrong	The initial historical model recommended raising prices on a category where every past increase had coincided with a supply shortage. It looked like customers were insensitive to price; they had had no alternative
The fix	Test, don't infer, where the cost of being wrong is high and a test is affordable. Historical elasticity became a way to choose what to test, not what to do
The lesson	Observational elasticity is Chapter 31's warning in its most expensive form. The two categories where the test contradicted the model paid for the whole program

Case 5 • Fraud detection at a bank

A composite; representative numbers.

The problem as stated	"Catch more fraud"
The decision that changes	Which transactions go to a 40-person review team that can handle about 3,000 cases a day
The data	Millions of transactions a day, a fraud rate near 0.1%, and labels that arrive between two days and three months later, when a customer disputes a charge
The approach	Gradient boosting on transaction, account, and device features, with rules kept alongside the model for the patterns that must be caught regardless. The metric is precision at a fixed review capacity , not AUC
The architecture	Streaming scoring under 100 ms → rules and model combined → a ranked review queue sized to capacity → analyst decisions feeding back as labels → daily retraining
The result	For the same 3,000 daily reviews, roughly double the fraud caught compared with rules alone, and a false-positive rate low enough that customer friction fell
What went wrong	A model retrained on analyst decisions started learning the analysts' blind spots: patterns they had never been shown stayed invisible, and its confidence in what it already caught grew (Chapter 56's feedback loop)
The fix	A small random sample of <i>unreviewed</i> transactions sent to analysts every day, purely to discover what the queue was missing. It costs review capacity and it is the only thing that breaks the loop

The lesson	The constraint was never the model; it was 3,000 reviews a day. Every design decision followed from a fixed amount of human attention, which is the same constraint as Chapter 58's approval queue
-------------------	---

Case 6 • Logistics routing for a distribution fleet

A composite; representative numbers.

The problem as stated	"Optimize our delivery routes"
The decision that changes	The order and grouping of 60 to 90 drops across 12 vehicles, planned the evening before
The data	Order addresses of varying quality, vehicle capacities, driver shifts, and travel times that a mapping API could supply but that nobody had ever measured against reality
The approach	A vehicle routing solver, which is operations research rather than machine learning, with a learned model used only to predict actual service time at each stop
The architecture	Orders → address geocoding and cleaning → solver run at 18:00 → routes to the drivers' app → actual timings captured → service-time model retrained monthly
The result	Roughly 12% fewer kilometres on the routes that were accepted
What went wrong	The first month's routes were rejected by the dispatch supervisor almost daily. The solver did not know that one customer only accepts deliveries before 10 a.m., that a particular lane is impassable for the large vehicles after rain, or that two drivers are not sent to the same neighborhood because of a long-standing dispute
The fix	Two weeks sitting with the supervisor, writing down constraints that existed only as habit, and adding an override mechanism that records the reason. Most of the "optimization" value came from encoding rules nobody had written down
The lesson	Optimization loses to undocumented constraints, every time. The interesting technical work was the solver; the valuable work was the interviewing

Case 7 • Customer support automation at a subscription business

A composite; representative numbers.

The problem as stated	"Deflect 50% of tickets with AI"
The decision that changes	Whether a customer gets an instant answer, a suggested article, or a human
The data	Four years of tickets with resolutions, a help center where a third of articles were out of date, and no record of which answers had satisfied anyone
The approach	Retrieval-augmented generation over the help center (Chapter 55), with grounding, citations, and a refusal path, plus an escalation button on every answer

The architecture	Question → hybrid retrieval → grounded answer with citation → thumbs up/down → escalation to a human with the transcript attached → refusals and thumbs-down into a content backlog
The result	About 35% of tickets resolved without a human, and, the number the team cared about more, satisfaction on resolved tickets equal to the human baseline
What went wrong	The first target was deflection rate, and deflection is trivially maximized by making the human path hard to find. Two weeks in, deflection was 48% and satisfaction had fallen sharply. Customers were giving up, not being helped
The fix	Change the metric to resolved-and-satisfied , make escalation one click, and treat every refusal as a documentation gap. Deflection fell to 35% and was worth having
The lesson	Deflection rate is a metric that can be gamed by hurting customers , which makes it a bad target and a fine diagnostic. Chapter 55's refusal path is what makes the honest version possible

Case 8 • Predictive maintenance on plant machinery

A composite; representative numbers.

The problem as stated	"Predict failures before they happen"
The decision that changes	Whether a maintenance team opens a machine during a planned stop
The data	Two years of sensor readings at one-second resolution, and 41 recorded failures , of which 12 had a usable timestamp
The approach	Anomaly detection first, because 12 labeled events cannot train a classifier. Later, a supervised model for the one failure mode that had enough examples, with everything else left as anomaly alerts for engineers to interpret
The architecture	Sensors → aggregation to minute level → anomaly scoring → alerts to the maintenance planner with the contributing sensors shown → engineer's verdict recorded, building the label set the project never had
The result	Two unplanned stoppages avoided in the first year, each worth several lakh rupees, and a labeled dataset that made year two's model possible
What went wrong	The initial alert threshold produced eleven alerts a week. Engineers investigated for a fortnight, found nothing most times, and stopped looking. The system was dead within a month and nobody said so
The fix	One alert a week, deliberately, even though it meant missing events, until trust was established. Alert volume was managed as a budget rather than a consequence of a threshold
The lesson	The label problem shapes everything. With 12 usable events the honest approach is anomaly detection plus a mechanism for creating labels, and the alert budget matters more than the algorithm

Case 9 • Order to cash, end to end

Riverstone's own, built across Chapters 12, 13, 16, 32, 54, 57 and 58.

The problem as stated	"We don't know where our orders are"
The decision that changes	Everything from what to mould this week to which customer gets chased for payment
The data	The ERP's orders, the CRM's leads, a warehouse's despatch notes, and forty purchase-order emails a day that were being retyped by hand
The approach	The whole book: a modeled warehouse (Chapters 12, 28, 32), a dashboard people actually use (Chapter 16), extraction of the emails (Chapter 54), operations around it (Chapter 57), and automated intake with a human in the loop (Chapter 58)
The architecture	Email → extraction → validation → ERP write or review queue; ERP and CRM → dbt models → marts → dashboard; the same marts feeding the monthly pack
The result	Order entry time from two hours a day to about half an hour; a single reported revenue number instead of two that disagreed; month-end reporting from three days to one morning
What went wrong	The straight-through automation looked excellent at 88% and was silently wrong on 19% of what it loaded (Chapter 58). It was nearly shipped on the strength of the first number alone
The fix	Assisted intake instead, full automation only for the email formats that scored perfectly, and a measured plan for extending that segment
The lesson	The project was never one project. It was six years of small pieces, and the value came from the least glamorous ones: a definition of net revenue everyone agreed on, a dashboard with the numbers people already argue about, and a queue with good reasons in it

What the nine have in common

Read across the cases rather than down them and five patterns appear, none of them technical.

Five things went wrong in nine different industries

	quality	demand	leads	pricing	fraud	routing	support	mainten.	order-cash
the stated problem was the wrong one	✓	✓		✓	✓	✓	✓		✓
data work dominated the schedule		✓	✓	✓	✓	✓		✓	✓
the constraint was human attention	✓	✓	✓		✓	✓	✓	✓	✓
the failure was organizational	✓	✓	✓	✓	✓	✓	✓	✓	✓
the metric had to change	✓	✓			✓		✓	✓	✓

Not one of these five is a modeling error, and not one would have been prevented by a better algorithm.

Most would have been caught in week two by sitting beside the person who had to use the thing.

Figure 59.2 – The same five things went wrong in nine different industries.

1. The stated problem was the wrong problem, in eight of nine cases. "Catch more fraud" was really "spend 3,000 reviews a day well". "Optimize our routes" was really "write down the constraints we have never written down". "Deflect 50% of tickets" was actively harmful as stated. The first deliverable of a data project is usually a better question, and the skill is asking *what decision changes?* until the answer is concrete (Chapter 5).

- 2. The data work dominated the schedule, and the model rarely took more than a tenth of it.** Lead scoring: six weeks of CRM cleaning, three days of modeling. Predictive maintenance: two years of sensor data and twelve usable labels. Demand planning: a promotions spreadsheet nobody owned. This is why Part II of this book is longer than Part V, and why "we'll get the data later" is the most expensive sentence in a project plan.
- 3. The binding constraint was almost always human attention.** Three thousand reviews a day, one alert a week, a coordinator's half hour, eleven investigations a fortnight, a planner's willingness to trust a number. Every one of these projects was really an exercise in **spending a fixed amount of human attention as well as possible**, and the threshold, queue, and alert-budget decisions that follow from that are more consequential than any hyperparameter.
- 4. The failures were organizational, and they were discovered late.** A model that learned the day shift. Planners who kept their spreadsheets. A dispatch supervisor rejecting routes. Analysts training the model on their own blind spots. Engineers who stopped investigating alerts. Not one of these is a modeling error, and not one would have been prevented by a better algorithm. Most would have been caught in week two by somebody sitting beside the person who had to use the thing.
- 5. The metric changed, and the project only worked afterwards.** Accuracy became cost per 1,500 parts. AUC became precision at fixed capacity. Deflection became resolved-and-satisfied. Forecast error became stock released. **The metric you start with is a hypothesis about what matters**, and being willing to replace it is a professional skill rather than an admission of error.

What changes by industry, and what doesn't

	Changes a lot	Stays the same
Data	volume, shape, quality, how much is unstructured	it is always messier than the demo, and the labels are always the scarce part
Regulation	banking and healthcare constrain what you may use and must explain; manufacturing rarely does	audit trails help you regardless
Latency	fraud in 100 ms, demand planning in a week	almost nothing needs to be as fast as people assume
Cost of an error	a wrong price, a wrong lorry, a wrong diagnosis	it sets the threshold, the review capacity, and whether to automate at all
Label delay	a click is instant; a maintenance failure takes months	it decides whether you can measure yourself, and how fast you can improve
Technology	solvers, gradient boosting, transformers, rules	the simplest thing that works is chosen far less often than it should be

The practical consequence: **an analyst moving between industries transfers most of what they know**. The domain takes six months to learn and the craft took years. Interviewers who claim otherwise are usually describing their own onboarding process rather than the difficulty of the work.

In the real world: how these projects actually start

None of the nine began as a project. They began as a complaint, and somebody translated it.

- *"The forecasts are rubbish"* → which decision do the forecasts feed, how wrong are they today, and what would a better one be worth?
- *"Sales are calling the wrong leads"* → how are leads prioritized now, what does a wasted call cost, and what does the CRM actually contain?
- *"We don't know where our orders are"* → who asks that question, how often, and what do they do differently when they know?

The translation is the job, and it is mostly asking **what decision changes** until somebody says something concrete. The pattern to watch for: a complaint about a *number* is usually a complaint about a *decision* that the number feeds badly, and the fix is often not a better number but a better path from the number to the decision.

Two more things are true of every project in this chapter:

- **The first version was smaller than anyone wanted.** One line, one category, one customer segment, one email format. The projects that tried to launch everywhere at once are the ones not written up here, because they didn't finish.
- **Somebody owned it after launch.** In each case a named person watched the numbers, worked the queue, and noticed when things drifted. The cases where that person did not exist are the ones that quietly stopped working: the alerts nobody investigated, the model trained on one shift, the forecast nobody used.

Common mistakes and how to spot them

Mistake	How it shows up in these cases	The habit that prevents it
Solving the stated problem	"Deflect 50% of tickets" achieved by hiding the human path	Ask what decision changes, until the answer is concrete
Budgeting for the model, not the data	Six weeks of CRM cleaning, unplanned	Assume the data work is most of the schedule, and say so up front
Ignoring how late the truth arrives	Fraud labels at 2 to 90 days; 12 usable failures in two years	Ask when ground truth lands, before choosing an approach
Inferring causality from history	Price rises that coincided with shortages	Test where you can; Chapter 31 where you can't
Leaking the future into training	Lead score built on fields filled in after conversion	Build training sets from point-in-time snapshots
Training on your own outputs	The fraud model learning the analysts' blind spots	A random sample of unreviewed cases, always
Setting alert volume by threshold	Eleven maintenance alerts a week, then none investigated	Treat alert volume as a budget

Mistake	How it shows up in these cases	The habit that prevents it
Launching everywhere at once	The cases that never shipped	One line, one segment, one format
No owner after launch	Models that quietly stopped working	Name the owner before the launch date
Measuring what is easy	Accuracy, AUC, deflection	Measure the decision, priced

Recap

- Every case in this chapter answers the same six questions: **what decision changes, what data exists, what is the simplest approach, where does the output go, what does an error cost, and who owns it.**
- **The stated problem was wrong in eight of nine cases.** Translating a complaint into a decision is the first deliverable.
- **The data work dominated,** and labels were the scarce resource in every case where they mattered.
- **Human attention was the binding constraint:** 3,000 reviews a day, one alert a week, a coordinator's half hour. Thresholds and queues follow from it.
- **The failures were organizational and late:** a model trained on one shift, planners who kept their spreadsheets, engineers who stopped investigating, analysts whose blind spots the model learned.
- **The metric changed in every successful case,** from something easy to measure to something that described the decision.
- **Industry changes the data, the regulation, and the cost of an error.** It does not change the craft, so a good analyst transfers.
- **Start small and name an owner.** Every case that worked did both; the ones that skipped either are not in this chapter.

Key terms

case study · project frame · decision-first framing · baseline · label scarcity · label delay · ground truth · operating point · review capacity · alert budget · deflection rate · resolved-and-satisfied · straight-through rate · silent error rate · feedback loop · blind spot sampling · point-in-time features · data leakage · confounded elasticity · experiment versus inference · undocumented constraints · adoption · override with reason · content backlog · ownership · phased rollout

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Part VII (Chapters 60–68)** is the professional side of these projects: stakeholders, storytelling, governance, ethics, and leading the work rather than doing all of it.
- **Chapter 5, Thinking Like an Analyst**, is where the translation skill in this chapter was first taught; reread it now that you have seen nine translations.
- **Chapter 53, 55 and 58** hold the runnable versions of cases 1, 7 and 9, with the numbers this chapter quotes.
- **Chapter 30 and 31** are behind case 4's central lesson: test where you can, infer carefully where you cannot.
- **Chapter 56 and 57** are the operations every one of these nine needed and only some of them had.
- **Chapter 75, Case Study & Take-Home Bank**, turns this frame into interview practice: you will be handed problems in exactly this shape and asked how you would approach them.
- **Chapter 79, The First 90 Days**, is what to do when one of these lands on your desk in a new job.